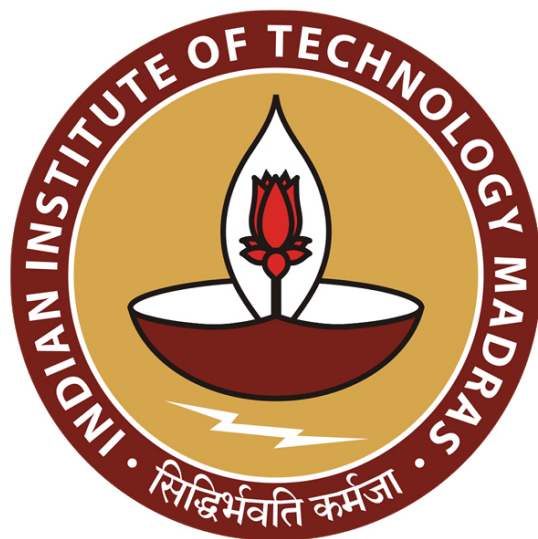




IIT Madras
ONLINE DEGREE

Machine Learning Techniques
Professor Arun Raj Kumar
Department of Computer Science & Engineering
Indian Institute of Technology, Madras
Kernel PCA

(Refer Slide Time: 0:14)

METHOD 2: MERCER'S THEOREM (Informal)

$\mathbb{R}^d: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a valid kernel

if and only if

(a) $\mathbb{R}$ is symmetric i.e., $\mathbb{R}(z_1, z_2) = \mathbb{R}(z_2, z_1)$

(b) For any dataset $\{x_1, \dots, x_n\}$, $K \in \mathbb{R}^{n \times n}$ where $K_{ij} = \mathbb{R}(x_i, x_j)$

All eigenvalues of K are non-negative.

$\phi(x)^T \phi(x)$
 $\phi(x) \phi(x)^T$

So, now putting all these things together, let us actually try to come up with the main algorithm that we wanted to do. We wanted to do PCA, but then where we have these nonlinear relationship between features, and we have discussed kernels as a trick or method to achieve that. So, let us put down this algorithm so that we see everything else in place.

(Refer Slide Time: 0:36)

KERNEL PCA

- Input - $\{x_1, \dots, x_n\}$ $x_i \in \mathbb{R}^d$, kernel $\mathbb{R}: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$
- Step 1: Compute $K \in \mathbb{R}^{n \times n}$ where $K_{ij} = \mathbb{R}(x_i, x_j)$
- Step 2: Compute $\beta_1, \dots, \beta_k$ eigen vectors
 $n \times 1 \dots n \times 1$ Eigen values

$$K_{ij} = k(x_i, x_j)$$

• Step 2: Compute $\beta_1, \dots, \beta_k$ Eigen vectors!
 $n\lambda_1, \dots, n\lambda_k$ Eigen values
 and normalize to get
 $\alpha_u = \frac{\beta_u}{\sqrt{n\lambda_u}}$

And that will make things clearer. So, this algorithm is what we are going to call as kernel PCA. So, what is the input, the input to this algorithm is as usual the data set $\{x_1, \dots, x_n\}$ where all x_i 's are in $\mathbb{R}^d$. Now in addition to this, we also have a kernel function given to you some kernel K , which is which takes any two d dimensional data points and then maps it to a real number. So, that is, we know that it is a valid kernel.

Let us say, we have tested Mercer's theorem and then argued that it's a valid kernel. So, that is given to us as input. Now, what is the step one? Well, step one, you will do the following. So, compute K . So, K in $n \times n$, where K_{ij} equals your kernel evaluated at the point x_i, x_j , so $\forall ij$. So, once we compute the kernel, then what is the next step? Well, we compute the Eigen vectors and Eigen values of this kernel matrix.

So, compute, L —let us call them β_1 to β_k , β_1 or β_k does not matter. And $n\lambda_1$ to $n\lambda_k$ as Eigen vectors and Eigen values of k . Once you have that, now, you can also normalize because we saw this last time, you need a normalization factor for β s because the length of α s is not same as one, which is what the length of β is. So, you need a normalization, you can normalize to get α_k is $\frac{\beta_k}{\sqrt{n\lambda_k}}$.

So, you do that, so you have got an α_k is for all k . Maybe I will put this as α_u . So, that just confused with other k that we have just an index. So, with how many of our Eigen vectors you want, so that that many alphas you get? So, what would be the step three?

(Refer Slide Time: 3:11)

$\sqrt{n} \lambda_n$

• Step 3: $w_R = \underline{\phi(x)} \alpha_R \rightarrow$ Defeats
be:

• we cannot "reconstruct" the Eigen V's
matrix

• But we can still compute the

• we cannot "reconstruct" the
matrix

• But we can still compute the

$$\begin{aligned}\phi(z_i)^T w_R &= \phi(z_i)^T \left(\sum_{j=1}^n \phi(z_j) \alpha_{Rj} \right) \\ &= \sum_{j=1}^n \alpha_{Rj} \phi(z_i)^T \phi(z_j) =\end{aligned}$$

सिद्धिर्भवति कर्मजा

Step 1: Compute $K \in \mathbb{R}^{n \times n}$ where
 $K_{ij} = k(x_i, x_j)$

Step 2: Compute $\beta_1, \dots, \beta_n$ Eigen vectors!
 $n \lambda_1 \dots n \lambda_n$ Eigen values
 and normalize to get
 $\alpha_u = \frac{\beta_u}{\sqrt{\lambda_u}}$

Step 3: $w_k = \phi(x) \alpha_k \rightarrow$ Defeats
 the

Well, in the original, when we are solving issue one, we said that step 3 now, is you get w_k you got that as

$x \alpha_k$ here, it is going to be $\phi(x) \alpha_k$. But there is a problem here. So, what is this telling us? Well, if you want w_k then you have to combine your data points in the higher dimensional space using the coefficients that you get from the Eigen computation of your kernel matrix.

But, the whole point of going through the kernel matrix was to avoid the computation of $\phi(x)$ in the first place, we know $\phi(x)$ cannot be computed. In some sense, if ϕ maps into an infinite dimensional space then you cannot even compute it let alone computational issues. It is simply impossible. So, now, doing this now, this defeats the purpose, so this is a bad idea, because it needs $\phi(x)$ which is what we are trying to avoid in the first place.

So, this looks like a bad idea, we should not compute w_k . So, what does that tell us? Well, this tells us the following. So, we cannot reconstruct the Eigen vectors of the covariance matrix, why because you need w_k and w_k needs $\phi(x)$, which we are not, let us say allowed to compute. So, that seems like an issue.

But, the good thing is that but we can still compute the compressed representation. So, compute or obtain the compressed representation. What do I mean by this? What I mean is that what you may not be able to do is to compute the Eigen vectors of the covariance matrix, but you can still compute the dot product of a or map data points on the Eigen vectors of the covariance matrix. What does that mean?

Well, how would we actually do the compression? So, if you go back to where we started, we said that we will find these directions and then project our data points along this direction to get these dot products. So, which means that after a in case, if I had the power to compute w_k , I would compute that and then I would do $\phi(x_i)$, which is the data point transpose w_k .

This is the number that I would like to store as the kth most important number corresponding to this data point x_i . But I cannot compute w_k . But do I have to is the question, so let us see,

this is just $\phi(x_i)^T \left(\sum_{j=1}^n \phi(x_j) \alpha_{kj} \right)$ we know is just some combination of the data points appropriately weighed by alpha k. So, alpha kj this whole thing then is just $\sum_{j=1}^n \alpha_{kj} \phi(x_i)^T \phi(x_j)$, I am just bringing the $\phi(x_i)$ inside.

Now, this just becomes $\sum_{j=1}^n \alpha_{kj} K_{ij}$. Essentially, this is k of kij . Well, that we already know because now, the dot product of the map data points on to the Eigen direction depends only on the dot products among the data points themselves, which I know already how to compute using my kernel function, I can compute this, this is a good. So, because we cannot of course, reconstruct the Eigen vector explicitly.

But we can compute the dot products or the projections onto this Eigen vectors in the high dimensional space. So, now, typically, the reason why you do kind PCA in general is that, you get these projections and then you map you throw away the original data points and only retain these projections along each of these data points.

That is where the dimensionality reduction happens. Original data point was 100 dimension but then you only care about the top 5. So, you only you map the 100 dimensional data into a 5 dimensional projection onto these 5 most important directions. So, that becomes a 5 dimensional representation of your 100 dimensional data.

Now, that you can do even here, so though you cannot compute on what it is being projected on, you can compute the value of this projection. That is what we are essentially saying here. So, the reason why this is that, so let me make a note here for downstream tasks. So, if you want to do this as a pre processing step, for some other, so once you learn that a presentation in a low dimensional space, as just the projections onto these Eigen directions.

You can use these projection values as, I mean, you can imagine that this is your data point. And then work with this for a downstream task, supervised learning task, which we will see later. So, this is for downstream tasks, this is usually good enough. So, you do not necessarily need to compute the Eigen vectors.

So, you would need the Eigen vectors only if you want to reconstruct the proxy. So, how would you reconstruct the proxy if I give you the projections? Well, you will multiply the projection with the corresponding Eigen vector and then add them up. So, for that, you need the Eigen vector. And if you need the Eigen vector, then you need $\phi(x)$, we have to let go of that. So, we would not be able to reconstruct the proxy in the high dimensional space.

But still, we will be able to store these projections and these predictions are typically good enough for downstream tasks. So, typically, what you do then is just, do not do step 3. So, you cannot do step 3. Instead, your step 3 would just be the following.

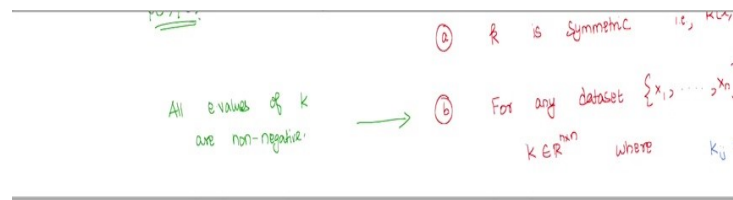
(Refer Slide Time: 9:42)

$$\begin{aligned} \phi(x_i) \omega_R &= \phi(x_i) \left(\sum_{j=1}^n \alpha_{Rj} \phi(x_j) \right) \\ &= \sum_{j=1}^n \alpha_{Rj} \phi(x_i) \phi(x_j)^T = \end{aligned}$$

Modified Step 3 :

Compute $\sum_{j=1}^n \alpha_{Rj} K_{ij} \neq R$

$x_i \in \mathbb{R}^d \rightarrow \left[\sum_{j=1}^n \alpha_{ij} K_{ij} \right]$



KERNEL PCA

Input - $\{x_1, \dots, x_n\}$ $x_i \in \mathbb{R}^d$; kernel $\text{K} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$

Step 1: Compute $\text{K} \in \mathbb{R}^{n \times n}$ where $K_{ij} = k(x_i, x_j)$
 "Center the kernel"

So, modified step 3, well, your modified step 3 would be to compute $\sum_{j=1}^n \alpha_{kj} K_{ij}$ for all k , so these are your numbers. So, basically you have your x , which is, f which is an $\mathbb{R}^d$. Now, what you are doing is you are mapping it to the following. So, you are mapping it to $\sum_{j=1}^n \alpha_{1j} K_{ij}$, $\sum_{j=1}^n \alpha_{2j} K_{ij}$, whatever top l that you want. So, this is what x_i , let us say, where x_i data point becomes this, basically.

So, of course, this K_{ij} is just the i th row of your kernel matrix, what you are doing is you are taking the i th row of your kernel matrix, which kind of tells you how similar is the i th data point to each of the other data points with respect to this kernel, and then you are weighting the similarities in different fashions given by these alphas, and then storing them as the most important, information corresponding to this data point.

So, from d dimension, you can reduce it to l dimension, where l is, I mean, you can either reduce or increase does not matter because you are going to a nonlinear higher dimensional space and then reducing the dimension. So, you might go from 2 to 100 dimensional space, and then reduce it to 10 dimensional space.

So, it might seem like you are actually increased from 2 to 10. But then this increase is necessary to capture the nonlinear structure of your data, it is a low dimensional representation of the maps data points, but which means it might actually increase the original dimension to a higher dimension, nevertheless, it is it might allow you to capture more complicated relationships.

So, this is the general idea of kernel PCA, there is one small important detail that we kind of missed out on that detail, comes somewhere here. In the original PCA, we assumed that we are given a data set and then you center this data set and then compute the covariance or the kernel matrix and then compute the Eigen vectors.

Now, here what we are seeing is that we are given a kernel, if the origin of the data is set to centered fine, we apply the kernel which means it is mapping it to a higher dimensional space using some mapping ϕ . But now, if you think of this data set $\phi(x_1)$,

$\phi(x_2)$

till $\phi(x_n)$ that may or may not be centered. So, that depends on your mapping kernel mapping.

It is a valid kernel, which means there is a ϕ , but then there is nothing that we are seeing that is that will make sure that the ϕ that we have which result in a map data, which is also centered, but then we need the center data only then our directions really make sense. So, we need to center the kernel. So, there is an extra step that is needed here, which is center we need to center the kernel.

So, which means we only have kernel cannot compute $\phi(x)$. So, is there a way to compute slightly different kernel, which will do the same mapping $\phi(x)$, but then after mapping $\phi(x)$, it will also center the data. So, is there a function that we can create which will so I give you a kernel, now that kernel will correspond to a ϕ map.

So, you want to apply that ϕ map, so you use the kernel, but then you also want to do the centering after you apply the phi map which means, can we mimic these two steps of going to a higher dimension and then doing the centering using just a single function kernel function, which is called as centering the kernel, so that that becomes important.

(Refer Slide Time: 14:04)

DETAILS: (Centering the kernel)

Given $K \in \mathbb{R}^{n \times n}$ $K_{ij} = k(x_i, x_j)$

Create a new kernel $K^c \leftarrow \text{center}$

$$K^c = K - \theta_i \mathbf{1}_j^T - \mathbf{1}_i \theta_j^T + P \mathbf{1}_i \mathbf{1}_j^T$$

where $\theta_i = \frac{1}{n} \sum_j k(x_i, x_j)$ and $P = \frac{1}{n}$

So, I will just give the details but then these are just details. But, I mean, for the sake of completeness, I am going to give this that you can indeed center the kernels. So, these are details, so how to centering the kernel? The good news is that you can center a kernel without explicitly computing ϕ . So, that is the most important news perhaps.

So, given kernel matrix K , which is an $n \times n$, so give you a kernel function from which you create a kernel matrix K for the data set that you have. So, you have n points in your data set. So, kernel matrix is an $n \times n$ matrix, where k_{ij} is just your kernel evaluated at x_i and x_j for all ij . Now create a new kernel.

So, you want to create a new kernel let us call this K^c , c for centered where K^c_{ij} is some function of K but then it does not use the explicit mapping $\phi(x)$. It can be done, we would not do the algebra here, but it's just for sake of completeness, I am saying this that this can be done. And it is good to know how this looks like, though it's not so important.

You take K_{ij} , and then you do the following. I will write it down and then make a comment $k^c_{ij} = k_{ij} - \theta_i \mathbf{1}_j^T + P \mathbf{1}_i \mathbf{1}_j^T$. So, basically, you take the original kernel value K_{ij} and then subtract with something, I mean, you need to do some subtraction. So, there is a mean subtraction happening in the higher dimensional space. And then we are trying to mimic what would happen if you do the mean subtraction and then do the dot product.

That is essentially what we are saying. So, on that dot product can be achieved by doing some manipulation of the original kernel itself. And that manipulation is, you multiply it with some value θ_i , which is just the average row sum corresponding to the i th data point. And then you do some manipulation, I mean, this $1_i, 1_j$ just refers to a vector, which just has one in the j th position and zero everywhere else.

And then that is an n dimensional vector. So, you basically, well, one can algebraically argue that this is exactly same as doing, mapping the following, so you have $\{x_1, \dots, x_n\}$, you apply your kernel, which will map it to $\{\phi(x_1), \dots, \phi(x_n)\}$. And then what you do is in the transform space, you do $\{\phi(x_1) - \mu, \dots, \phi(x_n) - \mu\}$, where μ is $\frac{1}{n} \sum_{i=1}^n \phi(x_i)$. Now, this is your new data point.

Now, if you take the dot product of any two data point here, well, that is going to look like $(\phi(x_i) - \mu)^T (\phi(x_j) - \mu)$. So, that would be $\phi(x_i)^T \phi(x_j) - \mu^T \phi(x_j) - \phi(x_i)^T \mu + \mu^T \mu$. So, there are four terms and those four terms correspond to these four guys, essentially, that is what it is.

So, this K_{ij} corresponds to this, we already know $\phi(x_i)^T \phi(x_j)$ is K_{ij} . Now, I mean, it is just a matter of algebra to verify that if I set θ_i and ρ as this, so then you will be able to get the other three terms as well. It is easy, I mean, it is just algebra, it is nothing, interesting going on here other than just saying that you can do this. So, that kind of completes the most important ideas for this algorithm.

(Refer Slide Time: 18:37)

KERNEL PCA

- Input - $\{x_1, \dots, x_n\}$ $x_i \in \mathbb{R}^d$; Kernel $\mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$
- Step 1: Compute $K \in \mathbb{R}^{n \times n}$ where

$$K_{ij} = k(x_i, x_j)$$

→ "Center the Kernel"
- Step 2: Compute $\beta_1, \dots, \beta_k$ Eigen vectors
 $\lambda_1 \geq \dots \geq \lambda_k$ Eigen values
 and normalize to get

$$u_k = \frac{\beta_k}{\sqrt{\lambda_k}}$$

$$\begin{aligned}\phi(x_i)^T \omega_R &= \phi(x_i)^T \left(\sum_{j=1}^n \phi(x_j) \alpha_{Rj} \right) \\ &= \sum_{j=1}^n \alpha_{Rj} \phi(x_i)^T \phi(x_j) =\end{aligned}$$

Modified Step 3: Compute $\sum_{j=1}^n \alpha_{Rj} K_{ij} \neq R$

$$x_i \in \mathbb{R}^d \rightarrow \left[\sum_{j=1}^n \alpha_{Rj} K_{ij}, \dots, \alpha_{Rj} \right]$$

DETAILS: (Centering the kernel)

And again, just to summarize, what we are doing now is that kernel PCA would be like this, you are given a kernel, compute the kernel and then you center the kernel, then get the Eigen vectors and Eigen values corresponding to the centered kernel, normalize it, get the α s and once you have the α s, do not construct the Eigen vectors instead, map it to these projections onto these Eigen vectors for each of the data points.

So, for every data point at the end, you have the top K projections and that projections, the set of projections is the representation for these data points as per kernel PCA. So, this is a very interesting technique. So, we have kind of used a lot of linear algebra here. But end of the day, all of this works out and we have been able to solve two important issues that we thought about for PCA, one is high dimensionality and the second is non linearity.

Surprisingly, the trick that solves both of them is the kernel matrix. You can map to higher dimension and then you can achieve nonlinear relationships as well. So, this kind of matrix is nothing specific to PCA you will encounter this object again and again, in this course, especially unsupervised learning as well, where the idea would be typically solve the problem for linear, when the data is linear has a linear relationship, which is easier problem solve. And now use the kernel trick to map it to a nonlinear problem.

So, here the kernel trick works because our whole algorithm is dependent only on pairwise products and not the exact data points themselves, if we know only the pairwise dot products, I can run this algorithm, which is essentially what the kernel matrix is doing in a high dimensional space. So, as we develop algorithms, we will also see if the algorithm depends only on the pairwise dot products.

If they do, then it suffices to kernelize this algorithm. Here in PCA, we observed that you can run PCA by only considering pairwise dot products and so we kernelized it and that resulted in kernel PCA. Similarly, we will kernelize slightly different other algorithms that we will encounter in this course.

But that is for later. For now, to summarize, we have a powerful unsupervised learning algorithm for representation learning and that is PCA and its kernel version, which is kernel PCA. Next time we will look at other types of unsupervised learning algorithms, including clustering and some kinds of estimation techniques. Thank you.

